

SEMAFOROS

Los **semáforos** son una herramienta fundamental en la programación concurrente, inventados por Edsger Dijkstra para resolver problemas de sincronización entre hilos o procesos. Para un principiante, se pueden visualizar como una variable entera especial que ayuda a controlar el orden de los eventos y el acceso a recursos compartidos.

A diferencia de un número entero normal, un semáforo tiene tres reglas estrictas:

1. **Inicialización:** Solo puedes darle un valor inicial al crearlo.
2. **Operación wait (esperar):** Decrementa el valor. Si el resultado es negativo, el hilo se bloquea y no puede continuar hasta que otro lo despierte.
3. **Operación signal (señalizar):** Incrementa el valor. Si hay hilos esperando, uno de ellos se desbloquea ("despierta") para continuar.

Estructura:

```
#define wait(s) atomic { s > 0 -> s-- }  
  
#define signal(s) s++
```

Patrones básicos de sincronización

Existen varios "patrones" o formas comunes de usar estos semáforos para que los programas funcionen correctamente:

1. Señalización (Signaling)

Es el uso más simple. Un hilo envía una señal a otro para indicarle que algo ha sucedido.

- **Para qué sirve:** Garantiza que una sección de código en el Hilo A se ejecute **antes** que una sección en el Hilo B.
- **Ejemplo:** Imagina que el Hilo A descarga un archivo y el Hilo B lo muestra en pantalla. B debe hacer un wait hasta que A haga un signal avisando que terminó.

2. Exclusión Mutua (Mutex)

Este patrón asegura que **solo un hilo a la vez** acceda a una variable compartida o a una "sección crítica".

- **Cómo funciona:** Se usa un semáforo inicializado en **1**. El primer hilo que llega hace wait, pone el semáforo en 0 y entra. Cualquier otro hilo que

llegue después se bloqueará en el wait hasta que el primero salga y haga signal.

- **Analogía:** Es como el "concho" en *El Señor de las Moscas*: solo el que tiene el concho puede hablar.

3. Cita (Rendezvous)

Se usa cuando dos hilos necesitan encontrarse en un punto exacto antes de que cualquiera de los dos pueda avanzar.

- **La regla:** El Hilo A debe esperar a B, y el Hilo B debe esperar a A.
- **Solución:** Se usan dos semáforos (ej. aLlego y bLlego) inicializados en 0. Cada hilo hace un signal para avisar que llegó y un wait para esperar al otro.

4. Multiplex

Es una generalización del Mutex, pero permite que hasta **N** hilos entren en una sección al mismo tiempo.

- **Uso real:** Imagina un club nocturno con una capacidad máxima. El semáforo se inicializa con el número de personas permitidas (N).
- **Mecánica:** Cada hilo que entra hace wait (consume un cupo) y cada uno que sale hace signal (libera un cupo). Si no hay cupos, los nuevos hilos esperan afuera.

5. Barrera (Barrier)

Se usa cuando tienes un grupo de hilos y quieres que todos terminen una tarea antes de que cualquiera comience la siguiente fase.

- **Funcionamiento:** Los primeros hilos se bloquean al llegar. Cuando llega el hilo número, este abre la "compuerta" y deja pasar a todos los demás.

Consideraciones importantes para principiantes

- **Interbloqueo (Deadlock):** Es un error común donde dos o más hilos se esperan mutuamente para siempre y el programa se congela. Esto ocurre frecuentemente si intentas hacer un wait sobre un semáforo mientras ya tienes bloqueado un Mutex.
- **Inanición (Starvation):** Ocurre cuando un hilo espera indefinidamente porque otros hilos siempre "le ganan" el acceso al recurso.
- **Semáforos fuertes vs. débiles:** Un semáforo **fuerte** garantiza que el hilo que lleva más tiempo esperando sea el primero en ser despertado (orden FIFO), mientras que uno **débil** no garantiza el orden.

Ejemplos:

1. Señalización (Signaling)

Este es el patrón más sencillo, donde un proceso envía una señal a otro para indicar que un evento ha ocurrido, garantizando el orden de ejecución (serialización).

```
#define wait(s) atomic { s > 0 -> s-- }

#define signal(s) s++

byte sem = 0; // Inicializado en 0 para bloquear al receptor

active proctype ProcesoA() {
    // ... realizar tarea a1 ...

    printf("Proceso A: tarea terminada\n");

    signal(sem); //signal: desbloquea a Proceso B [3]
}

active proctype ProcesoB() {
    wait(sem); // wait: espera la señal [1]

    printf("Proceso B: comenzando después de A\n");

    // ... realizar tarea b1 ...
}
```

2. Exclusión Mutua (Mutex)

Garantiza que solo un hilo a la vez acceda a una sección crítica. El semáforo se inicializa en 1, actuando como un "token".

```
#define wait(s) atomic { s > 0 -> s-- }

#define signal(s) s++

byte mutex = 1; // Inicializado en 1 para permitir el primer acceso [6]

int compartido = 0;

active [7] proctype Usuario() { // Crea 2 instancias del proceso
    do
        :: wait(mutex) // wait (P) [1]
            // Sección Crítica [8]
    od
}
```

```
    compartido++;  
  
    printf("PID %d en sección crítica. Valor: %d\n", _pid, compartido);  
  
    signal(mutex); // signal (V) [9]  
  
    // Sección No Crítica  
  
od  
  
}
```

3. Cita (Rendezvous)

Dos procesos deben encontrarse en un punto y ninguno puede avanzar hasta que ambos hayan llegado. Se requieren dos semáforos inicializados en 0.

```
#define wait(s) atomic { s > 0 -> s-- }  
  
#define signal(s) s++  
  
byte aLlego = 0;  
byte bLlego = 0;  
  
active proctype ProcesoA() {  
    printf("A llegó a la cita\n");  
    signal(aLlego); // signal: aviso que llegué [12]  
    wait(bLlego); // wait: espero a B [12]  
    printf("A continuando...\n");  
}  
  
active proctype ProcesoB() {  
    printf("B llegó a la cita\n");  
    signal(bLlego); // signal: aviso que llegué [12]  
    wait(aLlego); // wait: espero a A [12]  
    printf("B continuando...\n");  
}
```

4. Multiplex

Es una generalización del Mutex que permite que hasta N procesos entren en una sección al mismo tiempo. El semáforo se inicializa con la capacidad máxima (n).

```
#define wait(s) atomic { s > 0 -> s-- }

#define signal(s) s++

#define N 3 // Capacidad máxima

byte multiplex = N;

active [15] proctype Hilo() {

    do

        :: wait(multiplex) // wait: toma un cupo [14]

            printf("Hilo %d entró. Cupos restantes: %d\n", _pid, multiplex);

            // ... realizar tarea ...

            signal(multiplex); // signal: libera el cupo [16]

    od

}
```

5. Barrera (Barrier)

Se usa cuando un grupo de n hilos debe esperar a que todos lleguen antes de que cualquiera pueda continuar. Esta implementación utiliza un "molinete" (turnstile) para evitar bloqueos.

```
#define wait(s) atomic { s > 0 -> s-- }

#define signal(s) s++

#define TOTAL 3

byte count = 0;

byte mutex = 1;

byte turnstile = 0;

active [TOTAL] proctype Trabajador() {

    // Fase de llegada (Rendezvous)

    wait(mutex)
```

```
    count++;  
  
    if  
  
    :: count == TOTAL -> signal(turnstile); // Abre el molinete [19]  
  
    :: else -> skip  
  
    fi;  
  
    signal(mutex);  
  
    // Molinete: wait y signal consecutivos para dejar pasar uno a uno [19, 20]  
  
    wait(turnstile);  
  
    signal(turnstile);  
  
    printf("Trabajador %d pasó la barrera\n", _pid);  
  
    // ... Punto Crítico ...  
}
```

Ejercicios propuestos:

1. Patrón de Señalización (Signaling)

Contexto: Un sistema de procesamiento de datos en dos etapas.

Enunciado: Diseñe un programa donde el Proceso_A descargue un paquete de datos y el Proceso_B lo procese para su visualización. Debe garantizarse mediante un semáforo que la sección de código de visualización en B nunca ocurra antes de que la descarga en A haya finalizado por completo.

Reto: En Promela, verifique que no existe ningún estado donde B ejecute su tarea sin la señal de A. En Go, utilice un canal o la librería sync para emular el comportamiento del semáforo inicializado en 0.

2. Patrón de Exclusión Mutua (Mutex)

Contexto: Actualización concurrente de un saldo bancario.

Enunciado: Implemente un sistema donde 100 hilos intenten incrementar una variable compartida saldo en 1 unidad simultáneamente. Use un semáforo inicializado en 1 para proteger la sección crítica.

Reto: Demuestre mediante una aserción en Promela (assert) que el valor final de la variable es exactamente 100. En Go, ejecute el código sin el mutex para observar el error de concurrencia y luego corríjalo usando sync.Mutex.

3. Patrón de Cita (Rendezvous)

Contexto: Sincronización de fases en un juego multijugador.

Enunciado: Dos procesos, Jugador_1 y Jugador_2, deben encontrarse en un punto de control antes de iniciar una misión. Ninguno puede avanzar a la "Fase 2" del código hasta que ambos hayan llegado a la "Fase 1".

Reto: Implemente la solución usando dos semáforos (jugador1Llego, jugador2Llego) inicializados en 0. Verifique en Promela que es imposible que un proceso esté en la Fase 2 mientras el otro sigue antes de la Fase 1. Cuidado con el deadlock por el orden de las operaciones wait y signal.

4. Patrón Multiplex

Contexto: Gestión de aforo en una sala de servidores.

Enunciado: Una sala de servidores solo permite la entrada de un máximo de N técnicos simultáneamente para evitar el sobrecalentamiento. Si llega un técnico adicional cuando la sala está llena, este debe bloquearse hasta que alguien salga.

Reto: Configure el semáforo con un valor inicial igual a la capacidad máxima permitida. En Promela, modele una carga de trabajo que supere la capacidad y verifique que el número de procesos en la sección crítica nunca sea mayor a N. En Go, implemente esto usando un canal con buffer de tamaño N.

5. Patrón de Barrera (Barrier)

Contexto: Computación paralela por etapas (MapReduce simplificado).

Enunciado: Un grupo de n trabajadores realiza cálculos independientes. Al finalizar su parte, cada uno debe esperar en una barrera. Solo cuando el último trabajador llega, todos pueden proceder a la siguiente etapa de agregación de resultados.

Reto: Utilice una variable count protegida por un mutex y un semáforo de barrera que actúe como un "molinete" (turnstile). Los alumnos deben implementar una barrera reutilizable que permita sincronizar múltiples fases consecutivas sin causar interbloqueos si un proceso es más rápido que los demás. En Promela, verifique la ausencia de deadlocks usando el modelo de dos fases.